

Python 02

Elias Wachmann

2025

Content

1. Numpy
2. Generate random numbers
3. Matplotlib
4. File I/O
5. Numpy (extra)

Numpy

Numpy

Numpy is a python module that provides functions to work with arrays.

- Arrays are a data structure that can store multiple values of the same type.
- Arrays are indexed starting from 0.
- Arrays can be multidimensional.

Numpy examples

Quickstart Guide to Numpy

```
1 import numpy as np
2 # Create a 2D array of size 3x4
3 a = np.array([[1, 2, 3, 4], [5, 6, 7, 8],
4               [9, 10, 11, 12]])
5 print(a.shape)    # (3, 4)
6 print(a.ndim)     # 2
7 print(a.size)     # 12
8 print(a.dtype)    # int64
```

Numpy examples – Basics

The shape attribute of an array returns the dimensions of the array. In our example the array has 3 rows and 4 columns.

```
1 a = np.array([[1, 2, 3, 4], [5, 6, 7, 8],  
               [9, 10, 11, 12]])  
2 print(a.shape)    # (3, 4)
```

ndim specifies the number of dimensions of the array.

```
1 a = np.array([[1, 2, 3, 4], [5, 6, 7, 8],  
               [9, 10, 11, 12]])  
2 print(a.ndim)    # 2
```

Numpy examples – Basics

size returns the total number of elements in the array.

```
1 a = np.array([[1, 2, 3, 4], [5, 6, 7, 8],  
               [9, 10, 11, 12]])  
2 print(a.size)    # 12
```

Numpy examples – Zero & One arrays

zeros and ones can be used to create arrays filled with zeros or ones.

```
1 my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
2 new_array = np.array(my_list)
3 # 3 rows, 4 columns
4 all_zeros = np.zeros((3, 4))
5 # 3 rows, 1 column
6 all_ones = np.ones((3, 1))
```


Arange & Linspace

arange and linspace can be used to create arrays with evenly spaced values.

```
1 ints = np.arange(5, 42) # 5, 6, 7, ..., 41
2 linspace = np.linspace(0,5,11) # 0, 0.5, 1,
  1.5, ..., 5
3 fixed_width = np.arange(0, 10, 0.5) # 0,
  0.5, 1, 1.5, ..., 9.5
```

Arange: given step size

Linspace: given number of elements.

Numpy examples – Reshaping

reshape can be used to change the shape of an array.

```
1 square_matrix = np.array([[1, 2], [3, 4]])
2 column = square_matrix.reshape(4, 1)
3 row = square_matrix.reshape(1, 4)
4 print(column)
5 # [[1]
6 #   [2]
7 #   [3]
8 #   [4]]
9 print(row)    # [[1, 2, 3, 4]]
```

Numpy examples – Ravel

ravel can be used to flatten an array.

```
1 square_matrix = np.array([[1, 2], [3, 4]])
2 print(square_matrix)
3 # [[1 2]
4 #  [3 4]]
5 ravelled = square_matrix.ravel()
6 print(ravelled) # [1 2 3 4]
```

flatten is another function that can be used to flatten an array.

Benefits of working with numpy arrays

Numpy (mathematical functions) are evaluated on all elements of an array - in an element-wise fashion.

```
1 my_array = np.arange(1, 5, dtype=float)
2 el_wise = np.reciprocal(my_array)
3 print(el_wise) # [1.  0.5  0.33333  0.25]
4 # vs
5 my_array[0] = 1 / my_array[0]
6 my_array[1] = 1 / my_array[1]
7 my_array[2] = 1 / my_array[2]
8 my_array[3] = 1 / my_array[3]
9 print(my_array) # [1.  0.5  0.33333  0.25]
```

Generate random numbers

True randomness is not easy

- Computers are deterministic machines.
- Pseudo-random number generators (PRNG) are algorithms that can generate numbers that appear random.
- PRNGs are initialized with a seed value.
- The same seed value will result in the same sequence of random numbers.

Generate a random number

Create a Random Number Generator (RNG) [Line 2] and use various functions to generate random numbers from different distributions.

```
1 import numpy as np
2 rng = np.random.default_rng()
3 num_between_0_and_1 = rng.random()
4 num_between_5_and_42 = rng.uniform(5, 42)
5 num_from_std_gaussian = rng.normal(0, 1)
```

Generate a random number [Old way - Deprecated after Numpy 1.17]

rand can be used to generate a random number between 0 and 1.

uniform can be used to generate a random number between a given range.

```
1 from numpy import random
2 # low 0, high 1
3 rand_01 = random.random()
4 # low 5, high 42
5 rand_5_42 = random.uniform(5, 42)
6 # mean 0, std 1
7 rand_normal = random.normal(0, 1)
```


Matplotlib

Matplotlib - Basics

Matplotlib is a plotting library for the Python programming language. It provides various functions to create 2D, 3D plots and animations.

The pyplot module provides a MATLAB-like interface to the underlying plotting library.

Import the module with:

```
1 import matplotlib.pyplot as plt
```

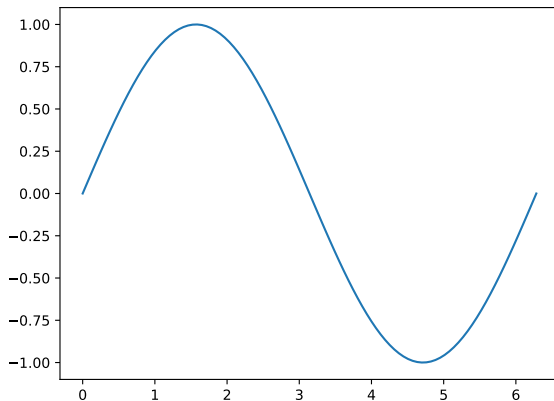
In the following examples, we will use the pyplot module (with alias `plt`) to create plots.

Matplotlib - Basic plotting

Let's create a basic plot of a sine function:

```
1 # Create a list of 100 evenly-spaced numbers  
   over the range 0 to 100  
2 x = np.linspace(0, 2 * np.pi, 100)  
3 y = np.sin(x) # Calculate sine for all x  
4 plt.plot(x, y) # Plot the sine for all x  
5 plt.show() # Display the plot
```

Matplotlib - Basic plotting output

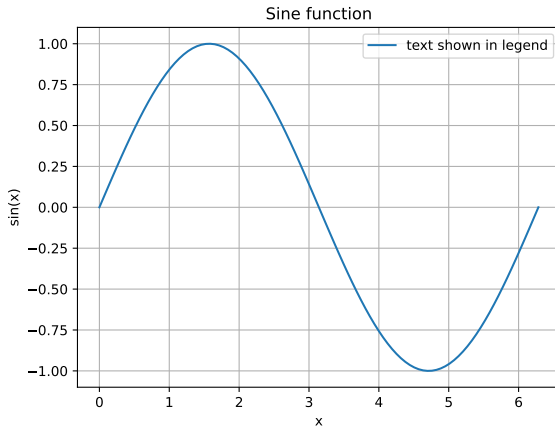


Matplotlib - Labels

We can add labels to the axes a legend and a title to the plot:

```
1 x = np.linspace(0, 2 * np.pi, 100)
2 y = np.sin(x)    # Calculate sine for all x
3 plt.plot(x, y, label="text shown in legend")
4 plt.xlabel("x")   # Label the x-axis
5 plt.ylabel("sin(x)") # Label the y-axis
6 plt.title("Sine function") # Add a title
7 plt.legend()      # Add a legend
8 plt.grid()        # Add a grid
9 plt.show()        # Display the plot
```

Documentation: [title\(\)](#), [xlabel\(\)](#), [ylabel\(\)](#), [legend\(\)](#)

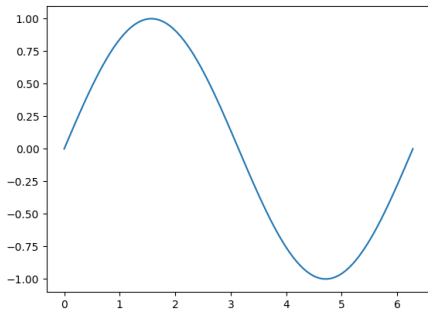
22
Matplotlib - Labels

Matplotlib - Save figures

To save plotted figures, use the `savefig()` function:

```
1 plt.savefig("path/to/save/plot/to.pdf")
```

Or use the floppy disk symbols in the pop-up windows



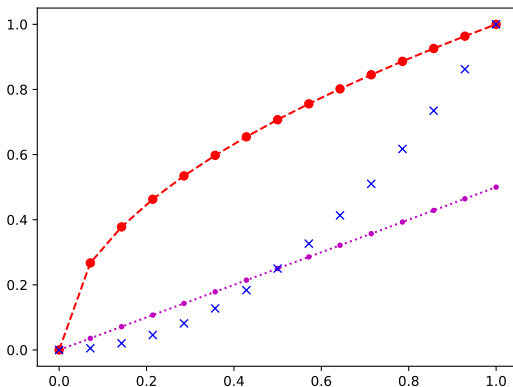
Matplotlib - Plotoptions

`plot()` function has many options to customize the plot. Format strings `fmt = '[marker][line][color]'` change the appearance of the lines:

```
1 x = np.linspace(0, 1, 15)
2 y1 = 0.5 * x
3 y2 = x**0.5
4 y3 = x**2
5 plt.plot(x, y1, ".:m")
6 plt.plot(x, y2, "o--r")
7 plt.plot(x, y3, "xb")
```


Matplotlib - Plotoptions

Using the plot options from the last slide we get:

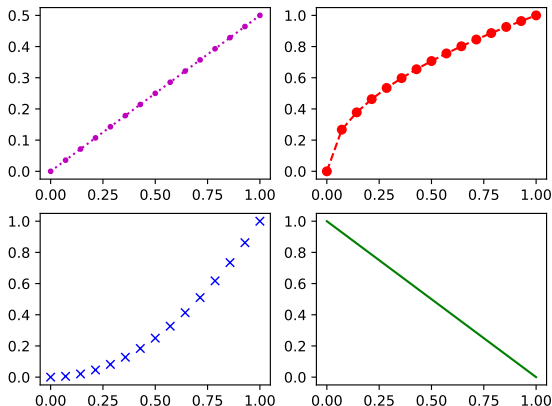


Matplotlib - Subplots

subplots() can be used to create multiple plots in one figure.

```
1 x = np.linspace(0, 1, 15)
2 y1 = 0.5 * x
3 y2 = x**0.5
4 y3 = x**2
5 y4 = 1 - x
6
7 fig, ax = plt.subplots(2,2)
8 ax[0,0].plot(x, y1, ".:m")
9 ax[0,1].plot(x, y2, "o--r")
10 ax[1,0].plot(x, y3, "xb")
11 ax[1,1].plot(x, y4, "g")
```

Matplotlib - Subplots



Matplotlib - Markers/Linestyles/Colors

Some markers: . (point), o (circle), + (plus), * (star), s (square), p (pentagon), h (hexagon 1), H (hexagon 2)

Line styles: - (solid line), -- (dashed line),
-. (dash-dot line), : (dotted line)

Colors: b (blue), g (green), r (red), c (cyan),
m (magenta), y (yellow), k (black), w (white)

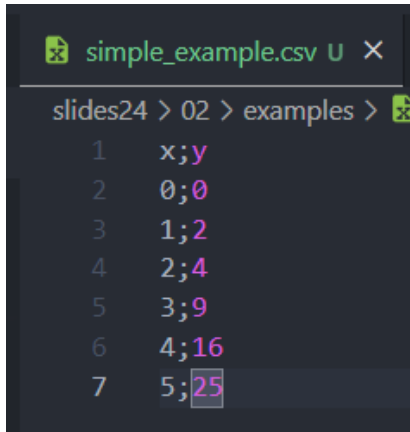
File I/O

Reading and writing files

loadtxt can be used to read data from a text file.

```
1 text = np.loadtxt('example_text.txt')
2 print(text) # prints the contents of the
   file
3 newtext = "Hello world!"
4 np.savetxt('example_text.txt', newtext)
5 text = np.loadtxt('example_text.txt')
6 print(text) # "Hello world!"
```

Reading csv files – input



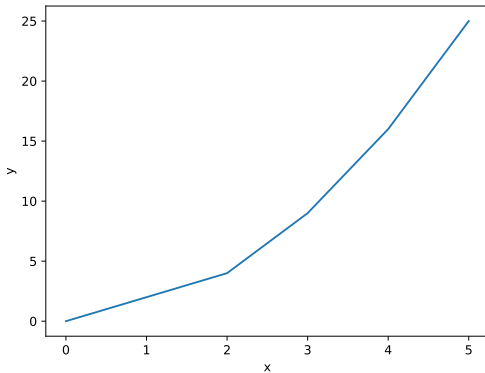
```
simple_example.csv U X
slides24 > 02 > examples > 
1      x;y
2      0;0
3      1;2
4      2;4
5      3;9
6      4;16
7      5;25
```

Reading csv files

genfromtxt can be used to read data from a csv file.

```
1 data = np.genfromtxt('slides24/02/examples/  
    simple_example.csv', delimiter=';')  
2 labels = np.genfromtxt('slides24/02/examples  
    /simple_example.csv', delimiter=';',  
    max_rows=1, dtype=str)  
3 print(labels)  
4 print(data)  
5 plt.plot(data[:,0], data[:,1])  
6 plt.xlabel(labels[0])  
7 plt.ylabel(labels[1])  
8 plt.show()
```


Reading csv files – output



Numpy (extra)

Ravel vs.Flatten

ravel returns a view of the original array. flatten returns a **copy** of the original array.

```
1 square_matrix = np.array([[1, 2], [3, 4]])
2 flattened = square_matrix.flatten()
3 ravelled = square_matrix.ravel()
4 square_matrix[0, 0] = 5
5 print(flattened)      # [1 2 3 4]
6 print(ravelled)       # [5 2 3 4]
```

Numpy examples – all & any

all and any can be used to check if all or any elements of an array comply with a condition.

```
1 x = np.array([1, 2, 3])
2 print(np.all(x != 0))    # True
3 print(np.any(x == 2))    # True
```